

dp 选讲

fzj2007

dp 是什么

动态规划是一种通过把原问题分解为相对简单的子问题的方式求解复杂问题的方法。—— Olwiki

从解决问题角度上来讲，dp 具有一些状态，每个状态可能通过决策（转移）到达其他状态。所以 dp 更像是一个自动机。那么就有了 dp 的几个关键性质：

- 相同子问题：小的问题与大的问题类似，每个状态信息的形式类似。
- 最优子结构：问题的最优解一定是通过子问题的最优解堆叠得到的。
- 无后效性：某个状态的决策确定后，后续的决策只与大当前状态有关，而不能与之前的决策有关。

dp 是什么

通常我们的目标是判定、计数和最优化。设计和优化 dp 的目标是使用尽量少的状态来表达过程中所有可能用到的结构与转移。

上面的描述与自动机是十分相似的。

例如，我们需要计数满足某条件的排列。我们令 $f_{i,s}$ 表示长度为 i ，排列是 s 的方案数，转移时在后面新增一个数并检查是否满足条件，这也是一种 dp，只是复杂度无法接受。如果此问题能够使用状压 dp 解决，我们可能会写出 f_s 表示当前填的数集合为 s 的方案数，然后枚举下一个数转移。相比前一个状态设计就有了明显优化。

所以 dp 的设计和优化本质是合并同类结构的过程。

完成一道 dp 题目的大致流程

- 首先需要像做任何一道题一样认真读题并整理出题目的大致模型。
- 经过一系列转化，确定要进行 dp 的对象。
- 设计 dp 状态来刻画问题。
- 优化 dp。

状态设计策略

此部分内容比较细碎，需要更多积累。这里可以给出大体的思考流程：

1. 从问题入手，先整体列出限制量和所求量。这是状态与转移中需要的关键信息。
 2. 尝试确定主要 dp 顺序。为达到此目的，可以观察限制量在某维度下的变化情况，更进一步地可以直接观察限制量之间的关系。对于一个看起来比较合理的 dp 顺序，限制量与所求量在此维度下应该依赖较少信息。
此部分可能有维度较多、维度需要转化等难点。在过程中要重点关注哪些变化难以求出的量。
 3. 在此顺序下添加需要的状态。如果发现某个量无法完整记录（例如排列信息等），尝试找到其他量与其的变化关系并优化记录方式。
- 如果实在无从下手，可以从暴力 dp 或特殊情况的 dp 入手进行扩展。

检验 dp 的正确性

主要可以从以下角度思考：

- 从单个可行解的角度入手，是否正确处理了每个解的贡献（尤其是计数问题中是否漏算、重算），不合法的解是否没有被计入答案。
- 从整体自动机考虑，是否有被错误合并的状态、是否有某个转移需要依赖其他信息。

ABC134F Permutation Oddness

定义长为 n 的一个排列 p 的奇妙度为 $\sum_{i=1}^n |i - p_i|$ 。

请求出长为 n 且奇妙度恰好为 K 的排列个数，对 $10^9 + 7$ 取模。

$1 \leq n \leq 50, 0 \leq K \leq n^2$ 。

Solution

首先，直接按位置进行序列上的 dp 大概是不可行的，因为我们不能记录填过了那些数，这就会导致我们难以计算当前位置的 $|i - p_i|$ 。

实际上 i 和 p_i 的位置是等价的，我们其实只关心哪两个数配对了。这引导我们可以同时从小到大填两个序列，在同一个配对点较小的数减掉贡献，较大的数加上贡献。

Solution

设 $f_{i,j,k}$ 表示当前两个序列都填了 $1 \sim i$ ，第一个序列前 i 个里面有 j 个未配对，当前奇妙度（包括提前计算的代价）为 k 的方案数。考虑给两个序列填入 $i + 1$ ，有下面几种情况：

- 两个序列的 $i + 1$ 配对，有 $f_{i+1,j,k} \leftarrow f_{i,j,k}$ 。
- 两个序列的 $i + 1$ 均与以前填过一半的位置配对，有 $f_{i+1,j-1,k} \leftarrow f_{i,j,k} \times j^2$ 。
- 有一个 $i + 1$ 与以前填过一半的位置配对，有 $f_{i+1,j,k} \leftarrow f_{i,j,k} \times 2j$ 。
- 两个都不与以前填过的配对且 $i + 1$ 不配对，有 $f_{i+1,j+1,k} \leftarrow f_{i,j,k}$ 。

最终答案即为 $f_{n,0,K}$ 。

时间复杂度 $\mathcal{O}(n^4)$ 。

LOJ6069 塔

现在有一条 $[1, l]$ 的数轴，要建立 n 座塔，塔编号为 $[1, i]$ 。每座塔的坐标均为整点且两两不同。

对于塔 i ，要求其与前与后面的塔的距离均至少为 i 。求有多少种建造方案，答案对 m 取模。

$$1 \leq n \leq 100, 1 \leq l, m \leq 10^9。$$

Solution

与连续段相关的 dp，与【摩天大楼】比较类似。

考虑最终塔的顺序，可以看做是一个排列。对于一个固定排列来说，我们可以求出其至少需要多长，然后乘一个组合数即可算出最终方案数。

现在考虑对每个长度求出有多少种排列。直接做序列 dp 是不太行的，因为相邻两个贡献是 max，而我们不可能知道那些数还没被用。

Solution

考虑从小到大插入每个数，记录当前连续段个数以及长度。设 $f_{i,j,k}$ 表示当前考虑到 i ，已经有 j 个连续段，最小距离为 k 的方案数，然后考虑 $i+1$ 的位置，有以下几种情况：

1. 在当前两个连续段中间，有 $f_{i+1,j-1,k+(i+1)*2} \leftarrow f_{i,j,k} \times (j-1)$ 。
2. 在当前一个连续段的一端，有 $f_{i+1,j,k+(i+1)} \leftarrow f_{i,j,k} \times 2j$ 。
3. 新开一个连续段，有 $f_{i+1,j+1,k} \leftarrow f_{i,j,k} \times (j+1)$ 。

时间复杂度 $\mathcal{O}(n^4)$ 。

ARC106E Medals

有 n 名员工，第 i 名员工会连续工作 a_i 天，然后连续休息 a_i 天，以此类推。

每天你可以选择从出勤员工中选择一人发放一枚奖牌。现在要让每位员工都获得至少 K 枚奖牌，求最少需要多少天。

$1 \leq n \leq 18, 1 \leq K, a_i \leq 10^5$ 。

Solution

首先答案不会超过 $2Kn$ 。

二分答案转化为判定性问题。然后可以看做一个匹配问题，人和天数分别是左部和右部，某人在某天上班是一条边，要求检查是否存在左部的 k 正则完美匹配。

左部点数很小，考虑 Hall 定理，要对每个左部集合 S 检查 $|N(S)| \geq K|S|$ 是否成立。即需要对于每个 S 求有多少天 $i \leq \text{mid}$ 满足该天上班的人的集合 $F(i)$ 与 S 有交。

转化为求无交的天数，只需要有多少 $F(i) \subseteq \overline{S}$ 。

使用高维前缀和即可，时间复杂度 $\mathcal{O}((2^n n + nK) \log K + n^2 K)$ 。

AGC009E Eternal Average

给定 n 个 0 和 m 个 1，重复进行以下操作：

- 从当前所有有理数中选出 k 个，删去这些数并添加他们的平均数。

保证无法操作时，恰好会剩下 1 个数。

求最后剩下的数的可能取值个数，对 $10^9 + 7$ 取模。

$1 \leq n, m \leq 2000, 2 \leq k \leq 2000, (k-1) \mid (n+m-1)$ 。

Solution

由于每次都选固定 k 个数，所以每次平均都会将数 $\div k$ 。

考虑将平均的操作过程建成树，每次删除 k 个点时建立 1 个新点作为这 k 个点的父亲，值为这 k 个点的平均数，则最终剩下的数即为树根的值。那么最终树上一个值为 x 的点
对根的贡献只与深度 d 有关，为 $\frac{x}{k^d}$ 。

对于两棵树来说，如果每层 1 的个数相同则最终根的值也一定相同。所以我们只关心每层有多少个 1。

但这样直接对树计数还是会算重，原因是一层可能出现大于等于 k 个 1。

Solution

考虑问题在哪里，实际上是可以将一层全为 1 的节点移动到另一个 1 下面。

所以我们砍掉子节点全为 1 的节点后计数。枚举砍掉了 t 次，则会使得 1 减少 $t(k-1)$ 个，但要求每层最多有 $k-1$ 个 1。容易证明这样能够不重不漏计算每个值（可以看成 k 进制小数序列）。

设 $f_{i,j}$ 表示深度为 i ，当前使用了 j 个 1 的方案数。枚举当前深度选择 c 个 1，则转移有

$$f_{i+1,j+c} \leftarrow f_{i,j}$$

最后枚举砍掉了 t 次，则限制为深度为 $\frac{n+m-1}{k-1} - t$ 且使用了 $m - t(k-1)$ 个 1。

时间复杂度 $\mathcal{O}(n^2)$ 。

CF1608F MEX counting

给定 n, k 以及长为 n 的数组 b ，求满足以下条件的数组 a 的个数：

- $\forall i(1 \leq i \leq n), 0 \leq a_i \leq n$ 。
- $\forall i(1 \leq i \leq n), |\text{mex}([a_1, a_2, \dots, a_i]) - b_i| \leq k$ 。

答案对 998244353 取模。

$1 \leq n \leq 2000, 0 \leq k \leq 50$ 。

Solution

对于 mex 来说，我们不关心每个数的出现次数，只关心每个数是否出现。更进一步地，对于给定的 $\text{mex} = k$ ，有 $< k$ 的所有数均出现， k 没有出现。现在的问题在于如何用尽量少的量来表达 $> k$ 的出现状态。

可以发现，我们在 mex 增加到更大之前，其实不关心更大的数的出现状态具体是什么，而只需要知道出现了多少个不同的数，在后面转移时再钦定这些数是什么就行了。所以可以设 $f_{i,k,j}$ 表示当前考虑到第 i 个位置， $\text{mex} = k$ 且 $> \text{mex}$ 有 j 个不同的数（但没有规定这些数是什么）的方案数。

Solution

按照第 $i + 1$ 个位置的填数情况分类讨论有

- mex 不变，填入比 mex 更大的，考虑填入的数有没有出现过可写出

$$\begin{aligned}f_{i+1,k,j} &\leftarrow f_{i,k,j} \times j \\f_{i+1,k,j+1} &\leftarrow f_{i,k,j}\end{aligned}$$

- mex 不变，填入比 mex 更小的。

$$f_{i+1,k,j} \leftarrow f_{i,k,j} \times k$$

- mex 改变， $i + 1$ 位置填 mex 后使得前缀 mex 增加了 c ，有转移

$$f_{i+1,k+c,j-(c-1)} \leftarrow f_{i,k+c,j-(c-1)} \times j^{c-1}$$

Solution

直接做时间复杂度是 $\mathcal{O}(n^4)$ 的，瓶颈在于第三类转移需要枚举 c 。

由 k 比较小，可以通过一系列下标平移将第二维和 c 的枚举优化到 $\mathcal{O}(k)$ 级别。状态数 $\mathcal{O}(n^2k)$ ，时间复杂度 $\mathcal{O}(n^2k^2)$ 。

考虑优化第三类转移。发现确定 mex 增大后转移与当前的 k, j 是没有直接关系的。采用分步转移的技巧，先将 k 平移到 $[-w_i, w_i]$ 内并转换到数组 g 中，然后后续 $k + j$ 相同的转移可以同时通过辅助数组 $g_{k+1, j-1} \leftarrow g_{k, j} \times j$ 维护。时间复杂度 $\mathcal{O}(n^2k)$ 。

dp 优化策略

主要可以分为状态设计优化和转移优化两部分。

下面有一些在优化开始之前常用的思考方式。

- 用得到的 dp 式子重新审视题目，观察是否有更简单的表达方式？
- 观察 dp 式子本身，寻找是否有组合意义？有没有冗余、重复、可以合并的状态或转移，以及是否有单调性？

状态设计优化

可以大概理解为对已经设计好的状态进行一些变换，使得最终状态数更少。此部分的大多数优化都较为明显，所以可能在状态设计的时候已经做过了。

- 压缩维度
- dp 值域状态换维：值域较小但某维度较大，且交换后能够得到答案。
- 调整下标记录信息：例如将 i 更换为 $\frac{n}{i}$ 等。
- 放宽限制：dp 的维度可能用于记录限制，若不满足某限制一定更劣（最优化问题）或不符合其他条件（计数问题），则可以删去该限制，如此可以减少 dp 维度。

转移优化

在状态数已经比较优的情况下，可能由于转移过多导致复杂度过高。

- 分步转移：在决策需要考虑多步过程（有些单步也可能能够强行拆成多步），多步的复杂度可能是每步复杂度的积。如果能够拆出中间层以过度，可能降低复杂度。
- 数据结构优化：主要有三种形式，一种是用来快速找决策点（例如 LIS 问题），一种是维护 dp 值（例如整体 dp），一种是快速完成多次转移（例如矩阵快速幂）。
- 结构优化：问题结构可能带来单调性等特殊性质，包括单调队列、斜率优化、决策单调性、凸优化等。
- 数学方法优化：这类有很强的数学指向性。常见的有矩阵快速幂、多项式优化等。

ICPC2025 上海 I. Round screws

- qoj15322

定义序列 a 的值 V_a 为

$$V_a = a_1 + a_n + \sum_{i=1}^{n-1} (a_i \oplus a_{i+1})$$

给定一个序列 a ，你可以进行任意次以下操作：

- 选择一个下标 $i (1 \leq i \leq n)$ 将 a_i 修改为任意非负整数，代价为常数 C 。

你需要最小化序列的值域操作产生的代价之和。即设进行了 p 次操作，则要求最小化 $pC + V_a$ 。

$2 \leq n \leq 10^5, 0 \leq C < 2^{19}, 0 \leq a_i < 2^{18}$ 。

Solution

首先每次找到增量最大的贪心是错的。

那么只能从最终结构来考虑，如果我们钦定了最终修改的数，那么对于一段极长连续的要修改的区间 $[l, r]$ ，将其全部修改为 a_{l-1} 一定不劣。因为此时这一段的贡献为 $a_{l-1} \oplus a_{r+1}$ ，而 a_{l-1}, a_{r+1} 二进制不同的位 i 必然至少有 2^i 的贡献，故上述修改方式可以达到下界。

先在序列开头和结尾分别插入一个 0 以避免边界情况。

设 f_i 表示考虑到位置 i 并且 i 不被修改的最小代价，则有

$$f_i = \min_j (f_j + (a_i \oplus a_j) + (i - j - 1)C)$$

得到了一个 $\mathcal{O}(n^2)$ 的做法，瓶颈在于转移时有一项 $a_i \oplus a_j$ 。

Solution

由于 a 值域较小，这里可以采用高低位分块的方式来优化。

记 x^1 为 x 的高 9 位， x^0 为 x 的低 9 位。维护一个类似分块的数据结构，共有 2^9 个块，每个块的长度为 2^9 ，均标号为 $[0, 2^9)$ 。

对于 j ，令其更新数据结构的第 $h = a_j^1$ 个块，对这个块的每个位置更新 $\text{val}_{h,x} \leftarrow \min(\text{val}_{h,x}, f_j + (-j - 1)C + (a_j^0 \oplus x))$ 。

对于 i ，枚举数据结构的每个块 h ，更新 $f_i \leftarrow \min(f_i, \text{val}_{h,a_i^0} + 2^9(h \oplus a_i^1) + iC)$ 。

实际上，将贡献拆成高低 9 位分别计算并均摊到查询和修改上，总复杂度为 $\mathcal{O}(n\sqrt{V})$ 。

CCPC2023 桂林 E. Prefix Mahjong

- qoj7681

给定一个正整数序列，判断其每个前缀是否为“和牌”。

多测， $1 \leq n, \sum n \leq 10^5, 1 \leq a_i \leq 10^9$ 。

称集合 $s = \{x, x, x\}$ 为刻子，集合 $\{x, x + 1, x + 2\}$ 为顺子，集合 $\{x, x\}$ 为雀头。

如果一个正整数集合可以被划分为若干个刻子、顺子与恰好一个雀头，则称其为“和牌”。

Solution

先考虑判定整个序列。从值域上考虑，对于当前值 i ，记录前一个和当前位置开始的顺子个数，用 dp 值表示这个状态是否合法。也就是 $f_{i,a,b,0/1}$ 表示当前考虑完值 i ， $i - 1$ 开始的顺子有 a 个， i 开始的顺子有 b 个，是否有雀头这个状态是否存在。

由于相同位置顺子达到 3 时可以转化成 3 个刻子，所以只需要保留 $0 \leq a, b < 3$ ，故总共只有 18 个状态。

由于值域比较大，但中间连续的空段是没用的，所以可以只保留值和值 +1 的数进行 dp。此时单次复杂度为 $\mathcal{O}(18n)$ 。

Solution

如果要对每个前缀考虑，则每次需要添加一个数，然后重新进行 dp。

发现上述 dp 从 i 到 $i + 1$ 的转移可以写成矩阵形式。以值域为下标建立线段树，每个区间维护对应的 dp 转移矩阵（也就是维护矩阵 $c_{x,y}$ 表示从区间开头的状态 x 是否能够到达区间结尾的状态 y ）。修改时只需要修改单个叶子，查询时拿出 $[1, m]$ 的转移矩阵即可得到最终 dp 值。

由于矩阵是 or 和 and 转移，可以压位优化，故时间复杂度为 $\mathcal{O}(18^2 n \log n)$ 。

云斗学院 #YDRS005E 将那朵云彩也跨越

对于一个 $n \times m$ 的网格，定义一种黑白染色合法，当且仅当用 1×2 的纸条能够不重叠且不超出网格边界地恰好覆盖所有白色格子，且恰好不覆盖任何黑色格子。

给定 q 个约束 (x_i, y_i, c_i) ，若 $c_i = 0$ 则表示 (x_i, y_i) 必须染成白色，否则表示 (x_i, y_i) 必须染成黑色。

求满足所有约束的合法染色方案数对 998244353 取模后的结果。

$$1 \leq n \leq 7, 1 \leq m \leq 10^{18}, 0 \leq q \leq 30.$$

Solution

m 非常大，看起来就像矩乘优化 dp。那么先考虑如何做正常 dp。

我们定义插头是长为 n 的 01 序列，其中 0 代表没有下一列延伸，1 代表向下一列延伸。

直接设 $f_{i,s}$ 表示当前考虑到第 i 列，当前插头状态是 s 是错误的，因为这样算出的将是纸条覆盖方案而不是合法的染色方案。我们可以发现这样无论如何记录都可能会算重。

实际上我们要关心当前状态能接受哪些插头，所以需要记录能接受插头的集合 S 。可以设 $f_{i,S}$ 表示当前考虑到第 i 列，当前能接收的插头集合为 S 的染色方案。但这样 S 的状态数高达 2^{2^7} ，无法接受。

Solution

接下来是类似 dp 套 dp 的过程，实际上是在 DFA 上 dp。

首先记录有相邻两个 1 的插头是无意义的。因为其一定不如将这两个位置改为 0 的插头，即所有需要这个插头状态的转移都可以通过改为 0 后的插头转移可达性，所以只需要保留 34 种插头（斐波那契数）。

接下来猜测合法的 S 状态数比较少，搜索处理出所有可达的 S 状态。具体地，维护一个队列表示当前可行状态，每次取出一个状态并枚举下一列的黑白填法，并找到其新状态并检查是否在队列中，依次类推直到无法继续拓展。

最终发现状态数仅 $n = 7$ 时仅 $k = 168$ （不同实现逻辑可能状态数不一样）。

Solution

这样已经可以做 m 较小的情况了。当 m 较大时，由于 q 很少，所以中间的大量转移是相同的。处理出 $i \rightarrow i + 1$ 中所有 S 的转移方程，可以写成一个矩阵。将限制按 y 排序，中间用矩阵快速幂加速转移 可以做到 $\mathcal{O}(k^3 q \log m)$ ，无法通过。

进一步使用向量乘矩阵的技巧，预处理 2 的幂次的转移矩阵，然后中间转移时采用二进制拆分并使用向量乘矩阵可以避免 q 内部的矩阵乘法。时间复杂度 $\mathcal{O}(k^3 \log m + k^2 q \log m)$ 。

CCPC2024 重庆 M. Median Replacement

- qoj9631

给定一个长度为 n 的序列 a ，你要进行若干次操作使得 a 中所有数均相等：

- 选择一段长为奇数且大于 1 的区间，将区间内所有数替换为其中位数。

设最终 $a_1 = \dots = a_n = x$ ，则定义序列 a 的价值为所有可能的 x 中的最大值。

请求出所有满足 $\forall 1 \leq i \leq n, l_i \leq a_i \leq r_i$ 的序列 a 的价值之和。

对 $10^9 + 7$ 取模。

$1 \leq T \leq 10, 3 \leq n \leq 150, 1 \leq l_i \leq r_i \leq 10^9$ 。

Solution

题目步骤比较多，需要先对核心部分推一些性质。

先考虑如何求出一个给定序列的价值。这是一个经典套路，考虑二分答案 k ，将 $\geq k$ 的标记 1，将 $< k$ 的标记 0。可以发现答案 $\geq k$ 的充要条件就是存在三个相邻位置中有至少两个 1。

Solution

先考虑一个依赖值域的做法，枚举 k 并计算价值为 k 的序列的贡献。可以转而求价值 $\geq k$ 的减掉价值 $\geq k + 1$ 的贡献。

设 $f_{i,a,b,c}$ 表示当前考虑到位置 i ，最近两个标记分别为 a, b ，且是否存在相邻三个位置中至少两个 1。则有 $0 \leq a, b, c \leq 1$ 。转移的时候考虑当前位置的标记是什么，并根据 l_i, r_i 计算选择 0 和 1 分别有多少方案即可。

Solution

但值域非常大， n 却非常小。注意到我们可以将题目中的数离散化成若干区间，则当 k 在其中一个区间 $[L, R]$ 波动时，每个 i 选择 0 或 1 的方案数分别可以看做一个一次多项式。具体有

- 若有 $L > r_i$ 则选择 0, 1 的多项式为 $r_i - l_i + 1, 0$ 。
- 若有 $R < l_i$ 则选择 0, 1 的多项式为 $0, r_i - l_i + 1$ 。
- 否则有 $[L, R] \in [l_i, r_i]$ ，选择 0, 1 的多项式为 $k - l_i, r_i - k + 1$ 。

根据上面的转移，任意 dp 值均最多是不超过 $n + 1$ 次的多项式，为了求出区间 $k \in [L, R]$ dp 值的和还要做一次前缀和。直接取 $0 \sim n + C$ 做 dp 并求前缀和，然后进行拉格朗日插值即可求出该多项式在 L, R 处的值。

时间复杂度 $\mathcal{O}(Tn^3)$ 。

CF1768F Wonderful Jump

给定 n 和长为 n 的序列 a 。

从位置 i 跳到位置 j ($1 \leq i < j \leq n$) 需要花费 $\min(a_i, a_{i+1}, \dots, a_j) \times (j - i)^2$ 枚金币。

对于 $k = 1, 2, \dots, n$ ，求出从位置 1 经过若干次跳跃后跳到位置 k 需要的最小金币总数。

$2 \leq n \leq 4 \times 10^5, 1 \leq a_i \leq n$ 。

Solution

很容易写出一个暴力 dp，设 f_i 表示从 1 跳到 i 的最小金币总数，每次枚举 $j < i$ 转移即可，接下来考虑如何优化这个转移过程。

Solution

首先可以发现这个 $(j - i)^2$ 看起来比较大，如果我们能分成小区间并且 min 不变就一定更优。那么如果有 $\min = a_k (i < k < j)$ 则选择 $i \rightarrow k \rightarrow j$ 一定比 $i \rightarrow j$ 更优，因为最小值不变但 $(k - i)^2 + (j - k)^2 < (j - i)^2$ 。

因此可能最优的转移一定有 a_i 或 a_j 是区间严格最小值。

Solution

另一方面，我们考虑如果每次都只走 1 步，那么从 i 走到 j 的金币数不会超过 $n \times (j - i)$ 。那么如果这样转移不劣必须有

$$\min \times (j - i)^2 < n \times (j - i) \Rightarrow \min \times (j - i) < n$$

考虑根号分治，对 $j - i \leq \sqrt{n}$ 的部分暴力转移；对 $j - i > \sqrt{n}$ 的部分必有 $\min \leq \sqrt{n}$ ，有以下两种情况

1. a_j 为最小值，则 a_j 最多只有 $\sqrt{n}$ 种取值。处理出来 $\leq \sqrt{n}$ 的每个数当前最后一次出现位置，枚举值然后转移即可。
2. a_i 为最小值，此时可以直接暴力枚举到前面第一个比 a_i 小的位置，对这些位置转移。对于每个位置来说，最多被后面 $\sqrt{n}$ 个数枚举到，所以复杂度是对的。

总复杂度 $\mathcal{O}(n\sqrt{n})$ 。